

Segmentación Retórica y Detección de Contribuciones en Artículos Científicos en Español

Un Enfoque Comparativo entre Encoders Especializados y Grandes Modelos de Lenguaje

Proyecto de Grado · Maestría en Inteligencia Artificial
Departamento de Ingeniería de Sistemas y Computación
Universidad de los Andes · 2026

- Github: https://github.com/camm93/proyecto_grado_maia
- www: <https://uniandes-maia-g2.duckdns.org>

Tabla de contenido

1. [Equipo](#)
2. [Resumen del proyecto](#)
3. [Demo en producción](#)
4. [Arquitectura de la aplicación](#)
5. [Estructura del repositorio](#)
6. [Requisitos del entorno](#)
7. [Instalación local](#)
8. [Despliegue en AWS](#)
9. [Configuración y parametrización \(.env\)](#)
10. [Credenciales de ejemplo](#)
11. [Ejemplos de uso](#)
12. [Modelos](#)
13. [Notebooks](#)
14. [Datos](#)
15. [Smoke test - Pruebas unitarias](#)
16. [Solución de problemas](#)
17. [Créditos y agradecimientos](#)
18. [Licencia](#)

1. Equipo

Grupo 2 — MAIA 2026

Integrante	Correo institucional
Santiago Ballesteros Pastrán	s.ballesteros@uniandes.edu.co
Edwin Alexander Cifuentes Bastidas	ea.cifuentes@uniandes.edu.co
Natalia Moncada Suárez	n.moncadas@uniandes.edu.co
Cristian Alexis Murillo Martínez	ca.murillom12@uniandes.edu.co
Juan Pablo Pérez Atehortua	jp.pereza12@uniandes.edu.co

Asesor: Rubén Manrique, Ph.D.

2. Resumen del proyecto

MAIA (*Modular AI Analysis*) es un sistema automatizado para la estructuración de textos científicos en español que aborda dos tareas complementarias mediante un enfoque de Inteligencia Artificial centrada en los datos (*Data-Centric AI*):

- **Tarea 1 (T1) — Segmentación retórica:** clasificación de fragmentos textuales en siete categorías funcionales: *Introducción* (INTRO), *Antecedentes* (BACK), *Metodología* (METH), *Resultados* (RES), *Discusión* (DISC), *Limitaciones* (LIM) y *Conclusiones* (CONC).

Etiqueta	Color	Hex	Sección
INTRO	 Azul	#2563EB	Introducción
BACK	 Violeta	#7C3AED	Antecedentes
METH	 Verde lima	#84CC16	Metodología
RES	 Verde	#16A34A	Resultados
DISC	 Teal	#0F766E	Discusión
CONTR	 Rojo	#DC2626	Contribución
LIM	 Magenta	#C026D3	Limitaciones
CONC	 Ámbar	#D97706	Conclusiones

- **Tarea 2 (T2) — Detección de contribuciones:** clasificación binaria que identifica si un fragmento contiene un aporte científico original. Se modela como tarea **transversal** e independiente de T1, dado que una contribución puede coexistir con cualquier categoría retórica.

El sistema integra tres familias de modelos —un heurístico léxico-posicional, el encoder especializado **SciBETO-large *fine-tuned***, y dos modelos generativos de gran escala (**Llama 3.1 8B-Instruct** y **GPT-4o-mini**)— y compara sus desempeños bajo condiciones equiparables sobre un *Gold Test Set* con doble anotación.

Resultados principales

Tarea	Mejor modelo	F1 Macro	Notas
T1 — Segmentación retórica	GPT-4o-mini (zero-shot)	0.447	SciBETO-large <i>fine-tuned</i> : 0.400 (mejor de pesos abiertos)
T2 — Detección de contribuciones	SciBETO-large (text-only)	0.852	IC 95% bootstrap: [0.793, 0.900]; AUC-ROC = 0.946

Contribución diferencial: evidencia empírica de que la curaduría de datos y la alineación de la ventana de contexto son tan determinantes como la elección de arquitectura para el análisis del discurso científico en español. El detalle metodológico y los hallazgos completos están en el artículo ([docs/Articulo_PLN_Grupo2_2026.pdf](#)).

3. Aplicación en producción

La aplicación está desplegada y accesible públicamente en: <https://uniandes-maia-g2.duckdns.org>

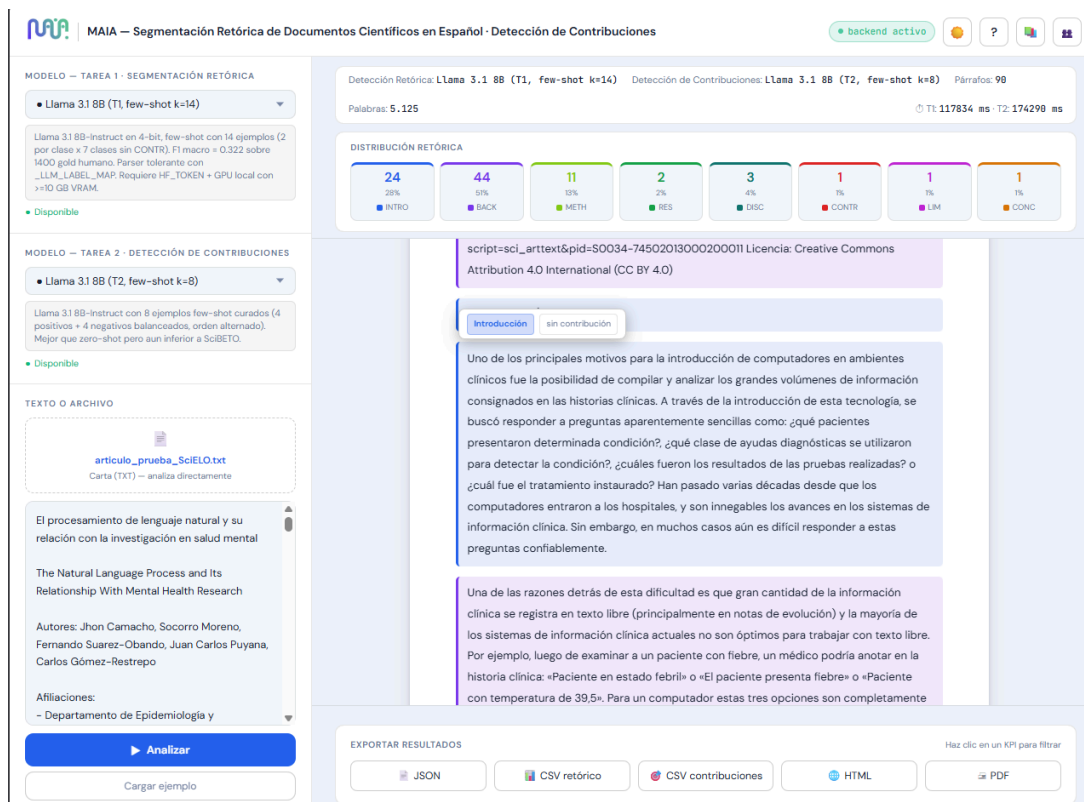


Figura 1. Aplicación en producción, resultado de segmentación retórica (Tarea 1).

Infraestructura: AWS EC2 **g4dn.xlarge** (GPU NVIDIA T4 16 GB) · Amazon Linux 2023 · Python 3.11 **Versión de la aplicación:** 7.8.1 **Certificado HTTPS:** Let's Encrypt con renovación automática (**certbot-renew.timer**) **Servicio:** **systemd** (**maia.service**) habilitado al arranque de forma persistente, **reverse proxy nginx**

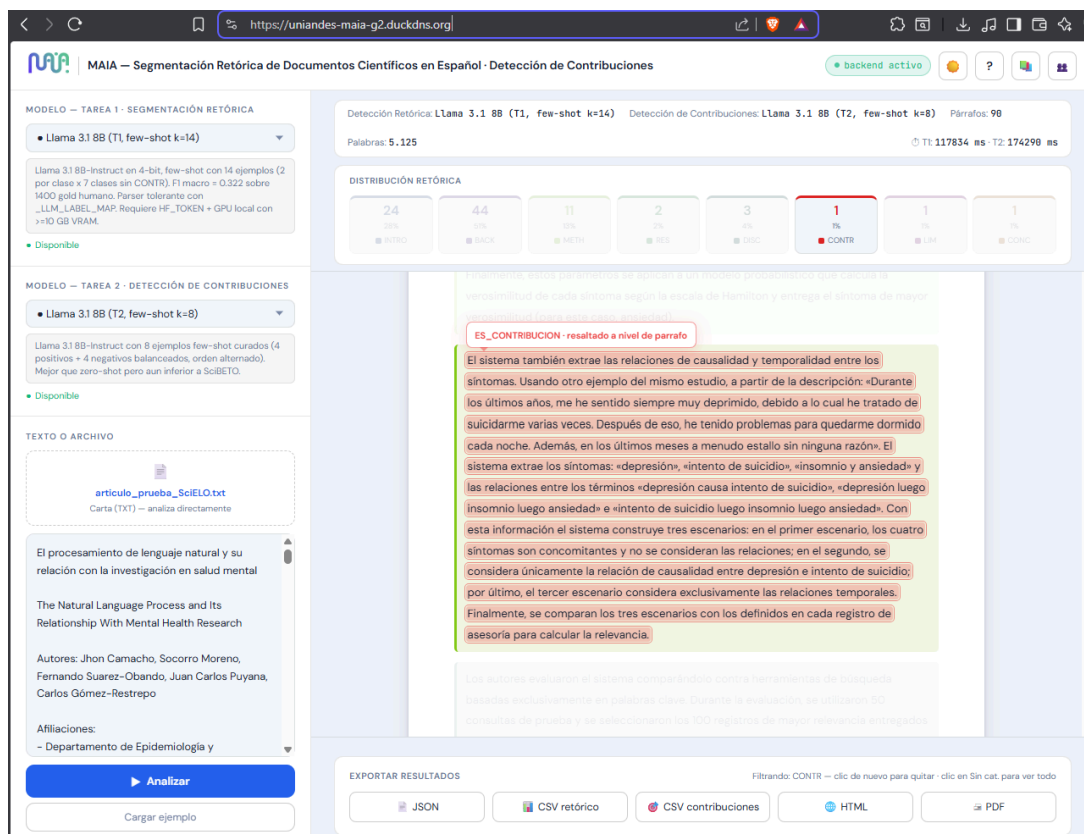


Figura 2. Aplicación en producción, resultado de detección de contribuciones (Tarea 2).

Recurso	URL
Aplicación web	https://uniandes-maia-g2.duckdns.org
Documentación interactiva de la API (Swagger UI)	https://uniandes-maia-g2.duckdns.org/docs
Schema OpenAPI (JSON)	https://uniandes-maia-g2.duckdns.org/openapi.json
Health check	https://uniandes-maia-g2.duckdns.org/health
Catálogo de modelos disponibles	https://uniandes-maia-g2.duckdns.org/api/models

Endpoints disponibles

Método	Ruta	Descripción
GET	/health	Estado del servicio, versión, modelos cargados
GET	/api/models	Catálogo completo de modelos T1 + T2 con descripciones
GET	/api/models/{model_id}/info	Metadata + métricas de evaluación de un modelo específico
GET	/api/few_shots	Inspección de los prompts few-shot (con SHA reproducible)
GET	/api/lexicon	Patrones léxicos del heurístico (regex, colores, prioridad)
POST	/api/segment	Segmentación retórica (T1)
POST	/api/contributions	Detección de contribuciones (T2)
POST	/api/extract-doc	Extracción de texto desde archivos .doc legados

Nota sobre extracción documental: la extracción de **PDF** y **DOCX** ocurre **client-side** en el frontend (con `pdf.js` y `mammoth.js`), por lo que no requiere viajar al backend. Solo los archivos `.doc` legados de versiones antiguas se procesan en el servidor con `antiword` (agradecimientos al proyecto: <https://github.com/grobian/antiword>).

Los ejemplos completos de uso de cada endpoint están en la sección [11. Ejemplos de uso](#).

Archivos soportados

Formato	Detección de columnas	Detección de tamaño de hoja
PDF	Sí (1, 2 o 3 columnas)	Sí (Carta, A4, Oficio, Folio)
DOCX / DOC	No necesario (Mammoth)	Sí (desde XML interno)
TXT	—	Siempre Carta

Exportación de resultados

- **JSON** — todos los campos (etiqueta, confianza, texto)
- **CSV** — tabla plana para Excel / pandas
- **HTML** — informe visual standalone con colores y highlights

4. Arquitectura de la aplicación

Vista general

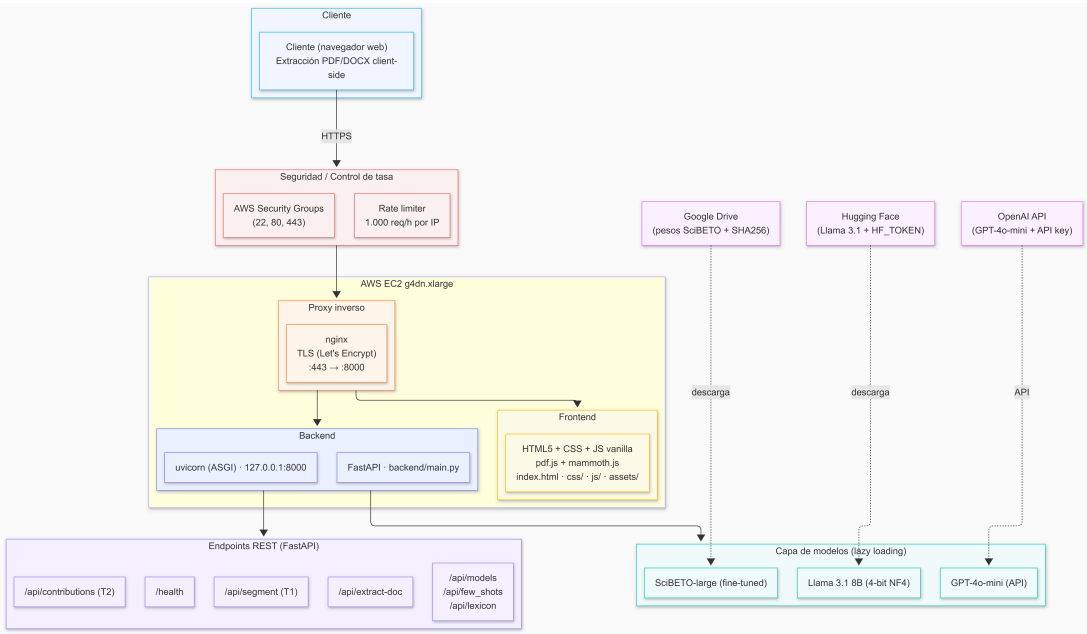


Figura 3. Arquitectura general de la aplicación.

► Ver código fuente del diagrama (Mermaid)

Stack tecnológico

Capa	Tecnología	Versión / detalle
Lenguaje	Python	3.11
Framework backend	FastAPI + uvicorn	latest stable
Encoders	transformers + torch	HF Transformers, PyTorch 2.x
LLM local	bitsandbytes (cuantización 4-bit NF4)	Llama 3.1 8B
LLM API	openai (Python SDK)	GPT-4o-mini con JSON mode
Extracción DOC	antiword (system package)	.doc legacy
Extracción PDF/DOCX	pdf.js + mammoth.js (client-side)	en el navegador
Frontend	HTML5 + CSS3 + JavaScript ES2020 (SPA)	—
Servidor web	nginx + Let's Encrypt (certbot)	reverse proxy + TLS
Orquestación	systemd (maia.service)	arranque al boot
Infraestructura	AWS EC2 g4dn.xlarge (GPU T4 16 GB)	Amazon Linux 2023
Espacio en disco	min 60GiB	SSD recomendado ~30GiB min para pesos de modelos

Familias de modelos disponibles

El selector del frontend expone tres familias para cada tarea. El heurístico opera como *fallback silencioso* cuando un modelo seleccionado no tiene pesos o claves disponibles. Los IDs listados son los que devuelve `GET /api/models`.

Tarea 1 — Segmentación retórica (7 clases)

Familia	ID en <code>/api/models</code>	Estrategia	F1 Macro	Notas
Heurístico	<code>heuristic</code>	Reglas léxico-posicionales	—	Siempre disponible, sin pesos
Encoder	<code>scibeto-es-t1</code>	Fine-tuning supervisado	0.400	Local, GPU, sin costo
LLM open	<code>llama-3.1-8b-t1</code>	Few-shot (k=14)	0.322	Local, GPU 4-bit, requiere <code>HF_TOKEN</code>
LLM open	<code>llama-3.1-8b-t1-zs</code>	Zero-shot	0.234	Local, GPU 4-bit
LLM API	<code>gpt-4o-mini-t1</code>	Few-shot (k=14) + JSON mode	0.414	OpenAI API, ~\$0.24 USD por 1.400 fragmentos
LLM API	<code>gpt-4o-mini-t1-zs</code> ★	Zero-shot + JSON mode	0.447	Mejor T1 , ~\$0.14 USD por 1.400 fragmentos

Tarea 2 — Detección de contribuciones (binaria)

Familia	ID en <code>/api/models</code>	Estrategia	F1 Macro	Notas
Heurístico	<code>heuristic</code>	Patrones explícitos	—	Siempre disponible
Encoder	<code>scibeto-es-t2</code> ★	Fine-tuning (text-only)	0.852	Mejor configuración global ; threshold óptimo = 0.525
Encoder	<code>scibeto-es-t2-ctx</code>	Fine-tuning (prefijo retórico)	0.821	Modelo de ablación
LLM open	<code>llama-3.1-8b-t2</code>	Zero-shot	0.393	Colapsa hacia clase negativa
LLM open	<code>llama-3.1-8b-t2-fs8</code>	Few-shot (k=8)	0.561	4 positivos + 4 negativos, orden alternado
LLM API	<code>gpt-4o-mini-t2</code>	Zero-shot	0.423	~\$0.001 USD por fragmento
LLM API	<code>gpt-4o-mini-t2-fs8</code>	Few-shot (k=8)	0.534	~2× costo del zero-shot

⚠ **IDs canónicos:** los modelos *few-shot* llevan el ID sin sufijo (`llama-3.1-8b-t1`, `gpt-4o-mini-t1`); los *zero-shot* llevan el sufijo `-zs`. Las variantes few-shot k=8 para T2 llevan `-fs8`. Esta convención puede parecer contraintuitiva pero es la que expone `/api/models`.

5. Estructura del repositorio

El repositorio del proyecto se encuentra disponible en: https://github.com/camm93/proyecto_grado_maia

proyecto_grado_maia/

├── README.md

├── app/

│ ├── backend/

│ │ ├── main.py

│ │ ├── model_loader.py

│ │ ├── schemas.py

│ │ └── requirements.txt

← este archivo

← aplicación funcional (deployable)

← FastAPI + lógica de modelos

← punto de entrada uvicorn

← carga perezosa de modelos

← Pydantic schemas

└─ Dockerfile · Dockerfile.gpu	
└─ api/	← routers: segment, contributions, extract, system
└─ core/	← servicios T1/T2: SciBETO, LLM, heurística, catálogo
└─ middleware/	← rate limiting por IP
└─ utils/	← model_sync (descarga desde Drive)
└─ frontend/	← UI estática (HTML + CSS + JS vanilla)
└─ index.html	
└─ css/styles.css	
└─ js/app.js	
└─ assets/	← logo, favicon
└─ scripts/	← automatización de deploy y operación
└─ download_models.sh	← descarga pesos desde Drive con verificación SHA256
└─ install_systemd.sh	← instala maia.service
└─ setup_https.sh	← configura Let's Encrypt
└─ update_duckdns.sh	← actualiza DNS dinámico
└─ smoke_test.sh	← validación post-deploy
└─ entrypoint.sh	← entrypoint Docker
└─ install_antiword.sh	← dependencia para .DOC legado
└─ maia.service	← unit file de systemd
└─ tests/	← pytest: tests unitarios y de integración
└─ test_catalog.py	
└─ test_contributions.py	
└─ test_extract_doc_endpoint.py	
└─ test_few_shots_endpoint.py	
└─ test_llm_service.py	
└─ test_model_loader.py	
└─ test_model_sync.py	
└─ test_rate_limit.py	
└─ test_t2_prefix.py	
└─ fixtures/sample.doc	
└─ start.sh · start.bat	← arranque local (Linux/Mac · Windows)
└─ docker-compose.yml	← orquestación CPU
└─ docker-compose.gpu.yml	← orquestación GPU (nvidia-container-toolkit)
└─ release_manifest.json	← versiones + SHA256 de pesos
└─ .env.example	← plantilla de configuración (copiar a .env)
└─ INSTALACION.md	← guía detallada de instalación
└─ DEPLOY_AWS.md	← guía detallada de despliegue AWS
└─ Notebooks/	← ★ Análisis, entrenamiento y pruebas
└─ T1_segmentacion_retorica/	← notebooks de Tarea 1 (Colab / Pro / Pro+)
└─ T2_deteccion_contribuciones/	← notebooks de Tarea 2 (Colab / Pro / Pro+)
└─ Modelos/	← ★ Pesos finales del proyecto
└─ URL a Google Drive (los pesos NO se versionan en Git; ver §12 y scripts/download_models.sh)	
└─ Datos/	← ★ Datasets T1 y T2
└─ dataset t1/	← Dataset T1 completo
└─ dataset t2/	← Dataset T2 completo
└─ docs/	← entregables académicos del curso
└─ images/	← archivos usados en el presente README
└─ articulo_prueba_SciELO.txt	← archivo de ejemplo de uso
└─ Articulo_PLN_Grupo2_2026.pdf	← artículo científico final

Notas sobre la estructura:

- La aplicación completa vive bajo `app/` para mantener separado el código *deployable* del material académico.
- Los **pesos no se versionan en Git** por su tamaño (~3.8 GB). Se distribuyen vía Google Drive con verificación SHA256, disponibles en:

 <https://drive.google.com/drive/folders/1zAdw781nvc46oWOWbkHLDuKNJG9uzh6G>

6. Requisitos del entorno

Software

Componente	Versión	Notas
Sistema operativo	Linux (Ubuntu 22.04 · Amazon Linux 2023 · RHEL 9) o Windows 10/11	macOS funciona con los comandos de Linux
Python	3.10+ (recomendado 3.11)	Verificar con <code>python3 --version</code>
<code>git</code>	cualquier versión reciente	Para clonar el repositorio
<code>unzip</code>	cualquier versión	Solo Linux/Mac
Conexión a Internet	requerida en el primer arranque	Descarga ~3.8 GB de pesos SciBETO + opcional ~16 GB de Llama

Hardware

Dos perfiles soportados según los modelos que se quieran ejecutar:

Perfil	Disco libre	RAM	GPU	Modelos disponibles
Completo	60 GB	16 GB (32 GB si Llama va en CPU)	NVIDIA T4 16 GB recomendada	SciBETO · Llama 3.1 8B · GPT-4o-mini
Reducido	10 GB	8 GB	No requerida	SciBETO · GPT-4o-mini (API)

Referencia de producción: en la instancia AWS EC2 `g4dn.xlarge` del equipo, `/data/app` ocupa aproximadamente **29 GB** con todo descargado y operativo (código + entorno virtual + 3 pesos SciBETO + cache de Llama 3.1 8B 4-bit).

Credenciales opcionales

Solo se requieren si se quieren habilitar los modelos LLM. Sin ellas, la heurística (búsqueda regex de marcadores retóricos) y SciBETO siguen funcionando.

Variable	Para qué	Cómo obtenerla
<code>OPENAI_API_KEY</code>	Habilitar <code>gpt-4o-mini-t1*</code> y <code>gpt-4o-mini-t2*</code>	https://platform.openai.com/api-keys
<code>HF_TOKEN</code>	Descargar Llama 3.1 8B (modelo <i>gated</i>)	1. Aceptar licencia en https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct 2. Generar token de lectura en https://huggingface.co/settings/tokens

Las dos claves se configuran en el archivo `.env` (ver §9). Si faltan, las variantes correspondientes quedan marcadas como **no disponibles** en el selector de la UI.

7. Instalación local

Esta sección cubre la instalación en una máquina de desarrollo (Linux, macOS o Windows). Para el despliegue en AWS con GPU, ver §8.

Paso 1 — Clonar el repositorio

```
git clone https://github.com/camm93/proyecto_grado_maia.git
cd proyecto_grado_maia/app
```

Todos los comandos siguientes se ejecutan desde `app/`.

Paso 2 — Arrancar la aplicación **Opción A — script automático de arranque (recomendada):**

```
# Linux / macOS
chmod +x start.sh
./start.sh

# Windows
start.bat
```

Nota: Este script se encarga de realizar de la automatización de:

- Creación/activación del entorno virtual python.
- Instalación de dependencias (antiword, pip upgrade, install requirements.txt, descargar pesos de drive, descarga de pesos llama),
- Ejecutar uvicorn y FastAPI

```
start.sh --help
# start.sh - Arranque del backend MAIA (Linux/macOS).
# =====
# Hace, en este orden:
# (pre) Instala antiword si falta (parser de .doc legacy, build ~30s)
#   - Linux:    sudo scripts/install_antiword.sh (idempotente)
#   - Otros:    warn y continua sin soporte .doc
# (a) pip install -r backend/requirements.txt (idempotente)
# (b) Sincronizacion + verificacion de pesos via model_sync
#   - 1er arranque: descarga 3 ZIPs desde Drive (~3.9 GB, ~5-8 min)
#   - Re-arranques: solo verifica SHAs (~5-10 s)
# (c) uvicorn backend.main:app --host 0.0.0.0 --port 8000
#
# Uso:
# ./start.sh                Arranque normal (dev local o produccion)
# ./start.sh --port 8080    Cambia el puerto (default 8000)
# ./start.sh --no-reload    Sin --reload (recomendado en produccion)
# ./start.sh --skip-install Omite el pip install (re-arranques rapidos)
# ./start.sh --skip-sync    Omite model_sync (asume pesos ya OK)
# ./start.sh --skip-antiword Omite check/install de antiword
# ./start.sh --verify-only  Solo verifica integridad y sale
```

```
# ./start.sh --help          Esta ayuda
#
# Variables de entorno relevantes (ver backend/utils/model_sync.py):
# MODELS_DIR                 default ./models
# MANIFEST_PATH              default ./release_manifest.json
# DOWNLOAD_FAIL_POLICY       abort | warn (default abort)
# T1_PRELOAD                 true | false (default true)
# T2_PRELOAD                 true | false (default true)
# T1_PRELOAD_ID              default scibeto-es-t1
# T2_PRELOAD_ID              default scibeto-es-t2
# =====
```

Si la ejecución de `start.sh` / `start.bat` termina con un mensaje en consola como el siguiente, la instalación fue correcta, saltar a **Paso 7. Verificar**

```
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      xxx.xxx.xxx.xxx:xxx - "GET /health HTTP/1.1" 200 OK
```

Paso 2 — Arrancar la aplicación: **Opción B — arranque manual:**

Crear el entorno virtual:

```
# Linux / macOS
python3 -m venv venv
source venv/bin/activate

# Windows (CMD)
python -m venv venv
venv\Scripts\activate.bat

# Windows (PowerShell)
python -m venv venv
venv\Scripts\Activate.ps1
```

Paso 3 — Instalar dependencias

```
python -m pip install --upgrade pip
pip install -r backend/requirements.txt
```

La instalación tarda 3–8 minutos según la conexión. Incluye PyTorch, Transformers, FastAPI, `bitsandbytes` (cuantización 4-bit) y el SDK de OpenAI.

Paso 4 — Configurar variables de entorno

```
cp .env.example .env
# Editar .env con tu editor preferido y agregar OPENAI_API_KEY / HF_TOKEN si aplica
```

La configuración mínima funciona con todos los defaults; solo es necesario editar `.env` si se quieren habilitar los LLM. La sección §9 documenta cada variable.

Paso 5 — Descargar los modelos

Los pesos se descargan automáticamente al primer arranque (si `DOWNLOAD_MODELS_ON_START=true` en `.env`, que es el default) o pueden descargarse manualmente antes:

```
bash scripts/download_models.sh
```

El script verifica el hash SHA256 de cada peso contra `release_manifest.json` y aborta si hay inconsistencia. Tiempo aproximado: 5–8 minutos en conexiones de 100 Mbps.

Paso 6 — Arrancar la aplicación

Opción A — script de arranque (recomendada):

```
# Linux / macOS
chmod +x start.sh
./start.sh

# Windows
start.bat
```

Opción B — arranque manual:

```
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
```

Paso 7 — Verificar

Abrir en el navegador:

- **Aplicación:** <http://localhost:8000>
- **Documentación API:** <http://localhost:8000/docs>
- **Health check:** <http://localhost:8000/health>

Una respuesta válida del health check se ve así (respuesta real del backend):

```
{
  "status": "ok",
  "version": "7.8.1",
  "endpoints": [
    "/api/segment",
    "/api/contributions",
    "/api/lexicon",
    "/api/models",
    "/api/models/{model_id}/info",
    "/api/few_shots"
  ],
  "models_t1": [
    "heuristic", "scibeto-es-t1",
```

```
    "llama-3.1-8b-t1", "llama-3.1-8b-t1-zs",
    "gpt-4o-mini-t1", "gpt-4o-mini-t1-zs"
  ],
  "models_t2": [
    "heuristic", "scibeto-es-t2", "scibeto-es-t2-ctx",
    "llama-3.1-8b-t2", "llama-3.1-8b-t2-fs8",
    "gpt-4o-mini-t2", "gpt-4o-mini-t2-fs8"
  ],
  "weights_on_disk": [
    "scibeto-es-t1", "scibeto-es-t2", "scibeto-es-t2-ctx"
  ],
  "t1_prediction_logging": false,
  "t2_prediction_logging": false
}
```

Si se quiere validar la instalación completa, ejecutar el smoke test (\$15):

```
bash scripts/smoke_test.sh http://localhost:8000
```

8. Despliegue en AWS

Esta sección reproduce el despliegue real del demo en <https://uniandes-maia-g2.duckdns.org>. Sirve como referencia para los evaluadores que quieran replicar el entorno de producción o como punto de partida para desplegar en una infraestructura propia.

8.1. Instancia recomendada

Instancia	GPU	VRAM	Costo aprox. (us-east-1)	Notas
g4dn.xlarge ★	T4	16 GB	~\$0.53/h on-demand · ~\$0.16/h spot	Usada por el equipo en producción
g5.xlarge	A10G	24 GB	~\$1.01/h	~2× más rápida en latencia
p3.2xlarge	V100	16 GB	~\$3.06/h	Sobredimensionada

- **AMI:** *Deep Learning Base GPU AMI (Amazon Linux 2023)* — viene con drivers NVIDIA y **nvidia-container-toolkit** preinstalados.
- **EBS:** mínimo 50 GB gp3 (sistema + venv + ~3.8 GB pesos SciBETO + ~16 GB cache Llama + margen operativo).
- **Security Group:** abrir puertos 22 (SSH, restringido a IP propia), 80 (HTTP, redirige a HTTPS) y 443 (HTTPS).

8.2. Receta de despliegue (TL;DR)

```
# 1. Conectarse a la instancia
ssh -i your-key.pem ec2-user@<ec2-public-ip>

# 2. Clonar el repositorio
git clone https://github.com/camm93/proyecto_grado_maia.git
cd proyecto_grado_maia/app

# 3. Configurar el entorno (Python 3.11 ya viene en Amazon Linux 2023)
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
```

```

pip install -r backend/requirements.txt

# 4. Configurar variables (agregar OPENAI_API_KEY y HF_TOKEN al .env)
cp .env.example .env
nano .env

# 5. Descargar pesos (~3.8 GB, 5-8 min en EC2)
bash scripts/download_models.sh

# 6. Configurar DNS dinámico (DuckDNS) y HTTPS (Let's Encrypt)
bash scripts/update_duckdns.sh
sudo bash scripts/setup_https.sh

# 7. Instalar como servicio systemd
sudo bash scripts/install_systemd.sh

# 8. Verificar
systemctl status maia
bash scripts/smoke_test.sh https://uniandes-maia-g2.duckdns.org

```

Si el smoke test devuelve 0 **fails**, el demo está operativo en la URL configurada.

8.3. Qué hace cada script

Script	Función
<code>scripts/download_models.sh</code>	Descarga pesos desde Google Drive, valida SHA256 contra <code>release_manifest.json</code>
<code>scripts/update_duckdns.sh</code>	Actualiza el registro DNS dinámico de DuckDNS apuntando a la dirección IP pública (WAN) de la instancia de EC2
<code>scripts/setup_https.sh</code>	Instala <code>certbot</code> , solicita certificado TLS de Let's Encrypt, configura <code>nginx</code> como reverse proxy y habilita renovación automática (<code>certbot-renew.timer</code>). Nota: El certificado SSL se emite para el dominio no para IP
<code>scripts/install_systemd.sh</code>	Copia <code>maia.service</code> a <code>/etc/systemd/system/</code> , hace <code>daemon-reload</code> , <code>enable</code> y <code>start</code> del servicio (persistencia de la aplicación en caso de reinicio del S.O.)
<code>scripts/install_antiword.sh</code>	Instala <code>antiword</code> (dependencia opcional para extracción de archivos <code>.doc</code> legados)
<code>scripts/entrypoint.sh</code>	Punto de entrada para ejecución dentro de Docker
<code>scripts/smoke_test.sh</code>	Suite de validación post-deploy (health, /docs, todos los endpoints)

8.4. Alternativa con Docker

Si se prefiere ejecutar dentro de contenedores en lugar de directamente sobre el host:

```

# Con GPU (requiere nvidia-container-toolkit)
docker compose -f docker-compose.gpu.yml up -d --build

# Sin GPU (modo CPU, más lento – solo recomendado para desarrollo)
docker compose up -d --build

```

```
# Seguir descarga inicial de pesos (~5-8 min)
docker compose -f docker-compose.gpu.yml logs -f maia
```

8.5. Operación y mantenimiento

Tarea	Comando
Reiniciar el servicio	<code>sudo systemctl restart maia</code>
Ver logs en vivo	<code>sudo journalctl -u maia -f</code>
Ver últimas 200 líneas	<code>sudo journalctl -u maia -n 200 --no-pager</code>
Renovar certificado TLS manualmente	<code>sudo certbot renew --dry-run</code> (validar) → <code>sudo certbot renew</code>
Actualizar el código	<code>git pull && sudo systemctl restart maia</code>
Espacio en disco	<code>df -h /data</code> (>8GB libres como mínimo, en caso contrario de reporta error; ver §16)

9. Configuración y parametrización (.env)

La aplicación se parametriza íntegramente mediante variables de entorno, leídas desde un archivo `.env` en la raíz de `app/`. La plantilla completa está en `.env.example`. Esta sección documenta cada bloque.

9.1. Crear el archivo

```
cd app/
cp .env.example .env
```

Importante: `.env` está en `.gitignore` y **nunca** debe versionarse, porque contiene secretos (API keys).

9.2. Variables generales

Variable	Default	Descripción
<code>MODELS_DIR</code>	<code>\$(pwd)/models</code>	Carpeta donde viven los pesos descargados. No cambiar salvo entorno Docker con mount específico.
<code>MANIFEST_PATH</code>	<code>\$(pwd)/release_manifest.json</code>	Ruta al manifest de versiones + hashes SHA256

⚠ Bug histórico: definir `MODELS_DIR=/app/models` cuando se ejecuta como servicio `systemd` causa `PermissionError` al intentar crear `/app/`. Dejar la variable comentada o apuntando a un path escribible por el usuario del servicio.

9.3. Tarea 1 — SciBETO (segmentación retórica)

Variable	Default	Descripción
<code>T1_DEVICE</code>	<code>auto</code>	<code>auto</code> · <code>cuda</code> · <code>cpu</code> . En <code>auto</code> usa GPU si está disponible.
<code>T1_BATCH_SIZE</code>	<code>32</code>	Tamaño de batch para inferencia. Reducir si hay <code>OOM</code> en GPU.
<code>T1_PRELOAD</code>	<code>true</code>	Cargar SciBETO-T1 al arranque (vs. lazy en primer request)

Variable	Default	Descripción
T1_PRELOAD_ID	scibeto-es-t1	ID del modelo a precargar
T1_LOG_PREDICTIONS	false	Persistir cada predicción en log (solo para análisis de errores)
T1_LOG_FULL_TEXT	false	Incluir el texto completo en el log de predicciones

9.4. Tarea 2 — SciBETO (detección de contribuciones)

Variable	Default	Descripción
T2_DEVICE	auto	Igual que T1_DEVICE
T2_BATCH_SIZE	32	Igual que T1_BATCH_SIZE
T2_PRELOAD	true	Precargar SciBETO-T2 baseline (text-only)
T2_PRELOAD_ID	scibeto-es-t2	ID del modelo baseline
T2_CTX_PRELOAD	true	Precargar también la variante de ablación con contexto
T2_CTX_PRELOAD_ID	scibeto-es-t2-ctx	ID del modelo de ablación
T2_THRESHOLD	0.525	Umbral de decisión binaria calibrado en validación
T2_LOG_PREDICTIONS	false	Persistir predicciones
T2_LOG_FULL_TEXT	false	Incluir texto completo en log

9.5. LLMs — GPT-4o-mini y Llama 3.1

Variable	Default	Descripción
OPENAI_API_KEY	(vacío)	Clave de OpenAI para habilitar todas las variantes gpt-4o-mini-*
GPT_MODEL_NAME	gpt-4o-mini	Modelo de OpenAI a usar
HF_TOKEN	(vacío)	Token de Hugging Face para descargar Llama 3.1 8B (modelo gated)
LLAMA_MODEL_NAME	meta-llama/Llama-3.1-8B-Instruct	Identificador del modelo en HF Hub
LLAMA_ENABLED	true	Habilitar/deshabilitar Llama globalmente
LLAMA_PRELOAD	true	Cargar Llama al arranque. Si false, se carga en el primer request (más lento).
HF_HOME	(auto)	Cache de modelos HF. Si vacío, se usa <project>/models/.hf_cache.
T2_LLM_LOG	false	Loggear respuestas crudas del LLM (debug, mucho ruido)

Tiempos de carga de Llama:

- GPU CUDA (g4dn.xlarge T4): **1–2 min** (4-bit, ~10 GB VRAM)
- CPU (Windows/Mac dev): **30–40 min** (FP16, ~16 GB RAM, descarga ~16 GB)

En máquinas con poca RAM (<32 GB), recomendado: LLAMA_PRELOAD=false en .env para arranque más rápido y carga diferida al primer uso.

9.6. Rate limiting

Variable	Default	Descripción
<code>RATE_LIMIT_PER_HOUR</code>	<code>1000</code>	Requests por IP por hora (sliding window). Aplica solo a <code>/api/</code> *
<code>RATE_LIMIT_WHITELIST</code>	(vacío)	CSV de IPs que bypassan el límite (health checks, monitores)

Para deshabilitar el rate limit en la práctica, setear un valor muy alto (ej. `999999`).

9.7. Descarga automática de pesos

Variable	Default	Descripción
<code>DOWNLOAD_MODELS_ON_START</code>	<code>true</code>	Descargar pesos automáticamente al arrancar si faltan
<code>DOWNLOAD_RETRY_ATTEMPTS</code>	<code>3</code>	Reintentos en caso de fallo de red
<code>DOWNLOAD_TIMEOUT_SECONDS</code>	<code>600</code>	Timeout por descarga (10 min)
<code>DOWNLOAD_FAIL_POLICY</code>	<code>abort</code>	<code>abort</code> → <code>sys.exit(1)</code> si falla / <code>warn</code> → loggea pero arranca con fallback heurístico

10. Credenciales de ejemplo

La aplicación **no requiere autenticación de usuario**: cualquier visitante de la URL pública puede usar la heurística y SciBETO sin credenciales. Las credenciales solo aplican a las APIs externas (OpenAI) y a la descarga de modelos *gated* (Hugging Face).

10.1. OpenAI API Key

Para qué: habilitar las variantes `gpt-4o-mini-t1`, `gpt-4o-mini-t1-zs`, `gpt-4o-mini-t2` y `gpt-4o-mini-t2-fs8`.

Cómo obtenerla:

1. Crear cuenta en <https://platform.openai.com/>
2. Cargar saldo mínimo (~\$5 USD alcanzan sobradamente para evaluar este proyecto)
3. Generar API key en <https://platform.openai.com/api-keys>
4. Copiar la clave en `.env`:

```
OPENAI_API_KEY=sk-proj-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Costo estimado para evaluación (precios actualizados a 18 de mayo de 2026):

- T1 zero-shot sobre 1.400 fragmentos del Gold: **~\$0.14 USD**
- T1 few-shot (k=14) sobre 1.400 fragmentos: **~\$0.24 USD**
- T2 sobre 170 fragmentos: **~\$0.09 USD**
- Procesar un artículo completo (~30 fragmentos): **~\$0.02 USD**

10.2. Hugging Face Token

Para qué: descargar Llama 3.1 8B-Instruct, que es un modelo *gated* (requiere aceptar licencia).

Cómo obtenerlo:

1. Crear cuenta en <https://huggingface.co/>

2. **Aceptar la licencia** en <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct> (esto es obligatorio; sin este paso el token no autoriza la descarga aunque sea válido)
3. Generar token de lectura en <https://huggingface.co/settings/tokens> (tipo *Read* es suficiente)
4. Copiar el token en `.env`:

```
HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

10.3. Modo sin credenciales

Si no se desea configurar ninguna credencial, la aplicación sigue funcionando con:

-  **Heurística** (todos los endpoints funcionales con búsqueda de expresiones regulares 'regex', latencia <50 ms)
-  **SciBETO-large fine-tuned** (T1 y T2, ambos baselines + ablación)
-  Llama 3.1 8B (queda como no disponible en el selector)
-  GPT-4o-mini (queda como no disponible en el selector)

Esto cubre los modelos de mejor desempeño en T2 (SciBETO F1 = 0.852) y el segundo mejor en T1 (SciBETO F1 = 0.400), por lo que la evaluación funcional del 80% del sistema puede hacerse sin gastar un dólar.

10.4. Seguridad de las credenciales

- **Nunca** commitar `.env` a Git (ya está en `.gitignore`).
- Rotar `OPENAI_API_KEY` si por accidente se filtra (la consola de OpenAI permite revocar e invalidar).
- El `HF_TOKEN` de tipo *Read* tiene riesgo mínimo (solo permite descargar modelos públicos que el usuario ya tiene acceso); aún así, no compartir.

 **Mejora por implementar:** migración a **AWS Secrets Manager** para gestión centralizada de credenciales en producción, con rotación automática y auditoría de acceso. Actualmente las credenciales viven en `/etc/maia.env` con permisos `600`, propiedad del usuario del servicio `systemd`.

11. Ejemplos de uso

La aplicación admite tres flujos: (a) interfaz web, (b) llamadas directas a la API y (c) integración programática vía cliente HTTP.

Todos los ejemplos de respuesta de esta sección son respuestas reales del backend en producción, capturadas al armar este README. No son ejemplos sintéticos.

11.1. Flujo recomendado en la interfaz web

1. Acceder a <https://uniandes-maia-g2.duckdns.org>
2. Cargar un documento (PDF, DOCX, DOC o TXT) mediante arrastrar-y-soltar o el botón **Subir archivo**, o pegar texto directamente en el área de entrada. Los PDF y DOCX se procesan en el navegador (no se envían al servidor); solo los `.doc` legados viajan a `/api/extract-doc`.
3. En el selector de modelos, elegir la familia para T1 y T2:
 - **Heurístico** — sin pesos, latencia <50 ms (ideal para vista previa)
 - **SciBETO-large** — encoder fine-tuned, mejor desempeño en T2 (recomendado)
 - **LLMs** — Llama 3.1 8B o GPT-4o-mini, con variantes zero-shot y few-shot
4. Hacer clic en **Analizar**
5. Revisar el resultado:
 - Cada párrafo aparece coloreado según su categoría retórica (T1)
 - Las contribuciones (T2) se resaltan dentro del párrafo

- El tooltip de cada párrafo muestra confianza T1 y T2
- La barra superior reporta latencia, número de párrafos y palabras

6. **Exportar** los resultados como JSON, CSV o HTML

11.2. Documento de prueba sugerido

Para validar el sistema con un artículo real de acceso público, usar:

Camacho, J., Moreno, S., Suarez-Obando, F., Puyana, J., & Gómez-Restrepo, C. (2013). El procesamiento de lenguaje natural y su relación con la investigación en salud mental. *Revista Colombiana de Psiquiatría*, 42(2), 227-233. DOI: 10.1016/S0034-7450(13)70011-8

Ejemplo disponible aquí: [docs/articulo_prueba_SciELO.txt](#).

Este documento expone una estructura retórica clara y contribuciones declaradas con marcadores típicos del español académico, lo que lo hace adecuado para evaluar cualitativamente ambas tareas.

11.3. Llamadas directas a la API

Todos los endpoints aceptan `application/json` y devuelven la misma. La documentación interactiva (Swagger UI) está disponible en `/docs` y permite hacer pruebas sin escribir código.

Health check

```
curl -s https://uniandes-maia-g2.duckdns.org/health | python3 -m json.tool
```

Respuesta real:

```
{
  "status": "ok",
  "version": "7.8.1",
  "endpoints": [
    "/api/segment", "/api/contributions", "/api/lexicon",
    "/api/models", "/api/models/{model_id}/info", "/api/few_shots"
  ],
  "models_t1": [
    "heuristic", "scibeto-es-t1",
    "llama-3.1-8b-t1", "llama-3.1-8b-t1-zs",
    "gpt-4o-mini-t1", "gpt-4o-mini-t1-zs"
  ],
  "models_t2": [
    "heuristic", "scibeto-es-t2", "scibeto-es-t2-ctx",
    "llama-3.1-8b-t2", "llama-3.1-8b-t2-fs8",
    "gpt-4o-mini-t2", "gpt-4o-mini-t2-fs8"
  ],
  "weights_on_disk": ["scibeto-es-t1", "scibeto-es-t2", "scibeto-es-t2-ctx"],
  "t1_prediction_logging": false,
  "t2_prediction_logging": false
}
```

Catálogo de modelos disponibles

```
curl -s https://uniandes-maia-g2.duckdns.org/api/models | python3 -m json.tool
```

Devuelve la lista de modelos T1 y T2 con **id**, **name**, **family**, **available**, **task** y **description**. El campo **available** indica si el modelo está utilizable (pesos en disco para SciBETO, credenciales válidas para los LLM).

Información detallada de un modelo

```
curl -s https://uniandes-maia-g2.duckdns.org/api/models/scibeto-es-t2/info | python3 -m json.tool
```

Devuelve catálogo + metadata + métricas de evaluación (arquitectura, num_labels, hidden_size, F1 Macro, AUC, IC bootstrap, threshold óptimo, kappa).

Inspección de prompts few-shot

```
curl -s https://uniandes-maia-g2.duckdns.org/api/few_shots | python3 -m json.tool
```

Devuelve los ejemplos exactos usados en los prompts few-shot de T1 (k=14) y T2 (k=8), con **sha256** del set completo para reproducibilidad. Esto permite a un evaluador auditar **qué se le está mostrando al LLM** sin necesidad de abrir el código.

Inspección del lexicón heurístico

```
curl -s https://uniandes-maia-g2.duckdns.org/api/lexicon | python3 -m json.tool
```

Devuelve los patrones regex, colores hex y prioridad de match del heurístico para T1 y T2. Permite auditar el baseline lingüístico y modificarlo si se necesita.

Segmentación retórica (T1)

Request:

```
curl -s -X POST https://uniandes-maia-g2.duckdns.org/api/segment \
-H "Content-Type: application/json" \
-d '{
  "text": "El procesamiento de lenguaje natural (PLN) trata de crear sistemas
informáticos que comprenden, procesan y generan lenguaje similar al humano. En este
artículo presentamos un sistema automatizado para la estructuración de textos científicos
en español.",
  "model_id": "scibeto-es-t1"
}' | python3 -m json.tool
```

Respuesta real:

```
{
  "model_id": "scibeto-es-t1",
```

```
"model_name": "SciBETO-ES T1 (SciBETO-large v12 fine-tuned)",
"mode": "encoder",
"elapsed_ms": 42,
"segments": [
  {
    "index": 0,
    "text": "El procesamiento de lenguaje natural (PLN) trata de crear sistemas
informáticos que comprenden, procesan y generan lenguaje similar al humano. En este
artículo presentamos un sistema automatizado para la estructuración de textos científicos
en español.",
    "char_start": 0,
    "char_end": 252,
    "word_count": 35,
    "relative_pos": 0.0,
    "rhetorical_zone": "Inicio (0-20%)",
    "t1_label": "METH",
    "t1_label_name": "Metodología",
    "t1_color": "#84CC16",
    "t1_confidence": 0.34,
    "low_confidence": true
  }
],
"summary": {
  "total_segments": 1,
  "t1_distribution": {"METH": 1},
  "avg_confidence": 0.34,
  "low_confidence_count": 1
}
```

Campos del segmento:

Campo	Tipo	Descripción
index	int	Posición del segmento (0-indexed)
text	string	Fragmento textual original
char_start, char_end	int	Offsets del fragmento en el texto completo
word_count	int	Número de palabras
relative_pos	float	Posición relativa en el documento [0.0, 1.0]
rhetorical_zone	string	Zona del documento (Inicio (0-20%) / Centro (20-80%) / Final (80-100%))
t1_label	string	Etiqueta predicha (INTRO/BACK/METH/RES/DISC/LIM/CONC)
t1_label_name	string	Nombre humano de la etiqueta
t1_color	string	Color hex para visualización
t1_confidence	float	Confianza de la predicción [0.0, 1.0]
low_confidence	bool	true si la confianza está bajo el umbral configurable

Detección de contribuciones (T2)

Request: los fragmentos deben venir del output de `/api/segment` (o construirse manualmente con al menos `index` y `text`).

```
curl -s -X POST https://uniandes-maia-g2.duckdns.org/api/contributions \
-H "Content-Type: application/json" \
-d '{
  "fragments": [
    {
      "index": 0,
      "text": "Este trabajo propone un nuevo método de clasificación retórica basado en
encoders especializados para el español científico."
    },
    {
      "index": 1,
      "text": "Los autores previos han abordado el problema desde múltiples enfoques sin
lograr consenso sobre la mejor arquitectura."
    }
  ],
  "model_id": "scibeto-es-t2"
}' | python3 -m json.tool
```

Respuesta real:

```
{
  "model_id": "scibeto-es-t2",
  "model_name": "SciBETO-large text-only (T2) -- recomendado",
  "mode": "encoder",
  "elapsed_ms": 45,
  "contributions": [
    {
      "index": 0,
      "is_contribution": false,
      "confidence": 0.5741,
      "label": "NO_ES_CONTRIBUCION",
      "t1_label": "",
      "t1_label_name": "",
      "t1_color": "#9CA3AF",
      "t1_confidence": 0.0,
      "rhetorical_zone": "",
      "rhetorical_context": "",
      "char_start": null,
      "char_end": null,
      "word_count": null,
      "relative_pos": null
    },
    {
      "index": 1,
      "is_contribution": true,
      "confidence": 0.7619,
      "label": "ES_CONTRIBUCION",
      "t1_label": "",
      "t1_label_name": "",
      "t1_color": "#9CA3AF",
      "t1_confidence": 0.0,
      "rhetorical_zone": "",
      "rhetorical_context": "Contribucion detectada (conf. 76%) en seccion '' (0%
conf. retorica) | ",
      "char_start": null,
      "char_end": null,

```

```
        "word_count": null,
        "relative_pos": null
    }
],
"summary": {
    "total_fragments": 2,
    "contributions_detected": 1,
    "contribution_rate": 0.5,
    "by_rhetorical_zone": {"": 1},
    "by_t1_label": {"": 1}
}
```

Schema completo del input (FragmentInput):

Campo	Tipo	Requerido	Default	Descripción
index	int	✓	—	Posición del fragmento
text	string	✓	—	Contenido textual
t1_label	string	✗	""	Etiqueta T1 predicha (enriquece contexto)
t1_label_name	string	✗	""	Nombre humano de T1
t1_color	string	✗	"#9CA3AF"	Color de T1
t1_confidence	float	✗	0.0	Confianza T1
rhetorical_zone	string	✗	""	Zona retórica del fragmento
char_start, char_end	int?	✗	null	Offsets en texto completo
word_count	int?	✗	null	Número de palabras
relative_pos	float?	✗	null	Posición relativa [0.0, 1.0]
low_confidence	bool	✗	false	Flag de baja confianza T1

Pasar el output completo de /api/segment como input de /api/contributions aprovecha el enriquecimiento retórico (los modelos T2 con prefijo de contexto lo usan).

11.4. Integración programática (Python)

```
import requests

BASE_URL = "https://uniandes-maia-g2.duckdns.org"

# Texto a analizar
texto = """
El procesamiento de lenguaje natural (PLN) trata de crear sistemas informáticos
que comprenden, procesan y generan lenguaje similar al humano. Este artículo
presenta un sistema automatizado para la estructuración de textos científicos.
"""

# 1) Segmentar retóricamente (T1)
segments_response = requests.post(
    f"{BASE_URL}/api/segment",
    json={"text": texto, "model_id": "scibeto-es-t1"},
```

```

).json()

# 2) Pasar los segmentos enriquecidos a T2
fragments = [
    {
        "index": s["index"],
        "text": s["text"],
        "t1_label": s["t1_label"],
        "t1_label_name": s["t1_label_name"],
        "t1_color": s["t1_color"],
        "t1_confidence": s["t1_confidence"],
        "rhetorical_zone": s["rhetorical_zone"],
        "char_start": s["char_start"],
        "char_end": s["char_end"],
        "word_count": s["word_count"],
        "relative_pos": s["relative_pos"],
        "low_confidence": s["low_confidence"],
    }
    for s in segments_response["segments"]
]

contrib_response = requests.post(
    f"{BASE_URL}/api/contributions",
    json={"fragments": fragments, "model_id": "scibeto-es-t2"},
).json()

# 3) Reportar resultados
for seg, contrib in zip(segments_response["segments"], contrib_response["contributions"]):
    flag = "★" if contrib["is_contribution"] else " "
    print(f"{flag} [{seg['t1_label']}:5s] conf={seg['t1_confidence']:.2f}] "
          f"{seg['text'][:80]}...")

```

11.5. Formatos de exportación

Formato	Contenido	Uso típico
JSON	Todos los campos (texto, etiqueta, confianza T1, confianza T2, posición)	Procesamiento posterior, integración con pipelines
CSV	Tabla plana: una fila por párrafo	Apertura en Excel / pandas para análisis cuantitativo
HTML	Informe visual <i>standalone</i> con colores y resaltado de contribuciones	Compartir resultados sin necesidad del backend

12. Modelos

Los pesos del proyecto se distribuyen mediante **Google Drive público**, con descarga automatizada y verificación de integridad SHA256 contra el archivo `release_manifest.json`. **No se versionan en Git** por su tamaño (~3.8 GB en total para los 3 modelos SciBETO).

12.1. Inventario de pesos

ID en /api/models	Tarea	Arquitectura	Tamaño (ZIP)	F1 Macro
scibeto-es-t1	T1 — Segmentación retórica (7 clases)	SciBETO-large fine-tuned	1.260 MB	0.400
scibeto-es-t2	T2 — Contribuciones (baseline text-only)	SciBETO-large fine-tuned	1.262 MB	0.852
scibeto-es-t2-ctx	T2 — Contribuciones (con prefijo retórico)	SciBETO-large fine-tuned	1.262 MB	0.821
llama-3.1-8b-*	T1 + T2	Llama 3.1 8B Instruct (4-bit NF4)	~16 GB	ver \$4
gpt-4o-mini-*	T1 + T2	API OpenAI (no se descarga)	—	ver \$4

Modelo base de los SciBETO fine-tuned: [Flaglab/SciBETO-large](#)

12.2. Carpeta pública de pesos

Todos los pesos del Grupo 2 viven en una carpeta pública de Google Drive:

 <https://drive.google.com/drive/folders/1zAdw781nvc46oWOWbkHLDuKNJG9uzh6G>

Archivo ZIP	URL directa	SHA256
scibeto-es-t1.zip	Drive	418471488863358e2310c999e3bce15dd8871670805745b66f013e0954707665
scibeto-es-t2.zip	ver release_manifest.json	(ver manifest)
scibeto-es-t2-ctx.zip	ver release_manifest.json	(ver manifest)

El archivo [release_manifest.json](#) en la raíz de [app/](#) contiene los hashes SHA256 y MD5 de cada ZIP **y de cada archivo individual** dentro del ZIP ([config.json](#), [model.safetensors](#), [tokenizer.json](#), [tokenizer_config.json](#), [training_args.bin](#)), lo que permite verificación granular de integridad incluso después de la extracción.

12.3. Descarga automática (recomendada)

La aplicación descarga los pesos automáticamente al primer arranque si [DOWNLOAD_MODELS_ON_START=true](#) (default en [.env.example](#)). Para forzar la descarga manualmente antes de arrancar:

```
cd app/
bash scripts/download_models.sh
```

El script:

1. Lee URLs y hashes SHA256 desde `release_manifest.json`
2. Descarga cada ZIP desde Google Drive (con reintentos según `DOWNLOAD_RETRY_ATTEMPTS`)
3. Verifica el hash SHA256 del ZIP descargado
4. Descomprime los archivos en `app/models/<id>/`
5. Verifica el hash SHA256 de cada archivo individual extraído
6. Aborta si hay inconsistencia (`DOWNLOAD_FAIL_POLICY=abort`)

Tiempo aproximado: **5–8 min** en conexión de 100 Mbps. La verificación SHA256 doble (ZIP + archivos individuales) protege contra descargas parcialmente corruptas o manipuladas.

12.4. Descarga manual

Si la descarga automática falla, se puede bajar cada ZIP manualmente desde el navegador y descomprimirlo en `app/models/<id>/`:

```
cd app/
mkdir -p models/scibeto-es-t1

# Bajar el ZIP desde el navegador:
# https://drive.google.com/file/d/1XZ_bNv4nNDS636caQg3z9rpNU04Gy-57/view

# Mover y descomprimir
mv ~/Descargas/scibeto-es-t1.zip ./
unzip scibeto-es-t1.zip -d models/scibeto-es-t1/

# Verificar integridad
sha256sum scibeto-es-t1.zip
# Esperado: 418471488863358e2310c999e3bce15dd8871670805745b66f013e0954707665
```

12.5. Estructura esperada tras la descarga

```
app/models/
├── scibeto-es-t1/
│   ├── config.json
│   ├── model.safetensors      ← ~1.3 GB
│   ├── tokenizer.json
│   ├── tokenizer_config.json
│   └── training_args.bin      (opcional, no requerido para inferencia)
├── scibeto-es-t2/
│   └── (misma estructura)
└── scibeto-es-t2-ctx/
    └── (misma estructura)
```

12.6. Verificación manual de integridad

```
# Verificar hash del ZIP completo
sha256sum app/models/scibeto-es-t1.zip
# Comparar con el campo "zip.sha256" en release_manifest.json

# Verificar hash de un archivo extraído
```

```
sha256sum app/models/scibeto-es-t1/model.safetensors
# Comparar con el campo correspondiente en "files[].sha256"
```

Si algún hash no coincide, **no usar el peso**: la descarga está corrupta o ha sido manipulada. Eliminar el directorio y volver a descargar.

13. Notebooks

La carpeta **Notebooks/** contiene los cuadernos de Jupyter usados durante el desarrollo del proyecto. **Cada integrante del grupo sube y mantiene sus propios notebooks**, organizados por tarea.

13.1. Organización

```
Notebooks/
├─ T1_segmentacion_retorica/   ← análisis, entrenamiento y evaluación de T1
└─ T2_deteccion_contribuciones/ ← análisis, entrenamiento y evaluación de T2
```

13.2. Convenciones

- **Nombres en snake_case**, descriptivos del paso del pipeline ML (EDA → Silver Labels → Train → Eval → Análisis de errores).
- **Resultados reproducibles**: fijar `random_state=42` / `seed=42` en todas las particiones y modelos, no obstante, el uso diferentes tipos de CPU/GPU puede incidir.
- **Sin secretos**: las claves API se cargan desde `.env` con `python-dotenv` o desde *Colab Secrets*, nunca hardcoded.

13.3. Entorno de ejecución

Los notebooks se desarrollaron en **Google Colab** (en sus tres tiers según el costo computacional):

Tier	GPU típica	Uso recomendado
Colab Free	T4 16 GB	EDA, evaluación cualitativa, inferencia con LLMs vía API
Colab Pro	T4 / V100 / A100 (disponibilidad)	Fine-tuning de SciBETO-large con datasets medianos
Colab Pro+	A100 40 GB / A100 80 GB	Fine-tuning de SciBETO-large completo (T1 y T2), evaluación de Llama 3.1 8B con cuantización

Requisitos para reproducir localmente (alternativa a Colab):

```
pip install jupyter pandas scikit-learn matplotlib seaborn transformers datasets accelerate
bitsandbytes
jupyter notebook
```

14. Datos

La carpeta **Datos/** contiene los **Datasets** completos liberados por el equipo bajo licencia [Creative Commons BY 4.0](#). El corpus crudo (~1.8M documentos del proyecto CORE) **no se versiona** por su tamaño y por restricciones del proveedor.

Política de liberación:

- **Test Sets** (anotación humana doble)

14.1. Contenido publicado

```
Datos/
├── dataset t1/      ← ★ Datasets T1 y T2
├── dataset t2/      ← Dataset T1 completo
└──                  ← Dataset T2 completo
```

14.2. Esquema de las categorías retóricas (T1)

Etiqueta	Nombre completo	Función discursiva	Marcadores típicos	Color
INTRO	Introducción	Plantea el problema y motiva el trabajo	"este trabajo aborda", "el objetivo de"	 Azul #2563EB
BACK	Antecedentes	Revisa literatura previa relevante	"trabajos previos", "se ha demostrado que"	 Violeta #7C3AED
METH	Metodología	Describe el procedimiento experimental	"se utilizó", "el corpus consistió en"	 Verde lima #84CC16
RES	Resultados	Reporta hallazgos cuantitativos	"se obtuvo", "la métrica alcanzó"	 Verde #16A34A
DISC	Discusión	Interpreta los resultados	"esto sugiere que", "en contraste con"	 Teal #0F766E
LIM	Limitaciones	Reconoce restricciones del trabajo	"sin embargo", "no fue posible"	 Magenta #C026D3
CONC	Conclusiones	Sintetiza hallazgos y trabajo futuro	"se concluye que", "en trabajo futuro"	 Ámbar #D97706

Los colores hex son los mismos que devuelve [/api/lexicon](#) y los que se aplican en la visualización del frontend.

14.3. Esquema de contribuciones (T2)

Etiqueta	Descripción	Patrones léxicos típicos
ES_CONTRIBUCION	El fragmento contiene un aporte original del autor	"se propone", "este trabajo presenta", "la principal contribución es", "se introduce", "nuestra propuesta"
NO_ES_CONTRIBUCION	El fragmento reporta hechos, antecedentes o descripciones sin novedad	(ausencia de los marcadores anteriores)

14.4. Formato CSV

Los Gold Test Sets usan separador `,` con encabezado y comillas dobles para textos. Columnas:

Columna	Tipo	Descripción
<code>id</code>	string	Identificador único del fragmento
<code>text</code>	string	Fragmento textual (250–1.000 palabras)
<code>label</code>	string	Etiqueta gold consensuada
<code>doc_source</code>	string	Procedencia del documento (CORE ID o tesis)
<code>position</code>	float	Posición relativa en el documento [0.0, 1.0]
<code>annotator_1</code>	string	Etiqueta del primer anotador
<code>annotator_2</code>	string	Etiqueta del segundo anotador
<code>agreement</code>	bool	<code>true</code> si ambos anotadores coinciden

14.5. Corpus completo (no publicado)

Conjunto	Tamaño	Notas
Corpus crudo CORE	1.812.557 documentos <code>.txt</code>	Provisto por el grupo FLAG; no redistribuible
Silver Labels T1	27.115 fragmentos	Generados por reglas heurísticas; uso interno
Silver Labels T2	1.600 entrenamiento + 400 validación	Patrones lingüísticos enriquecidos con T1; uso interno
Test Set T1	1.401 fragmentos	Liberado — $\kappa = 0.6714$
Test Set T2	200 fragmentos	Liberado — $\kappa = 0.6995$

Para acceso al corpus completo, contactar al grupo FLAG de la Universidad de los Andes.

15. Smoke test - Pruebas unitarias

El smoke test valida que **todos los endpoints respondan correctamente** después de un despliegue. Se ejecuta desde la raíz de `app/`:

```
bash scripts/smoke_test.sh https://uniandes-maia-g2.duckdns.org
```

(reemplazar la URL por `http://localhost:8000` para una validación local).

15.1. Qué prueba

Test	Endpoint	Criterio de aceptación
Health	<code>GET /health</code>	Status 200, <code>status == "ok"</code>
Docs UI	<code>GET /docs</code>	Status 200, HTML servido
OpenAPI schema	<code>GET /openapi.json</code>	Status 200, JSON parseable
Catálogo	<code>GET /api/models</code>	Status 200, ≥ 1 modelo disponible por tarea
Info modelo	<code>GET /api/models/scibeto-es-t2/info</code>	Status 200, <code>available == true</code>
Lexicón	<code>GET /api/lexicon</code>	Status 200, ≥ 7 clases T1

Test	Endpoint	Criterio de aceptación
Few-shots	GET /api/few_shots	Status 200, T1.k == 14 y T2.k == 8
Segmentación heurística	POST /api/segment	Status 200, ≥1 segmento devuelto
Segmentación SciBETO	POST /api/segment con scibeto-es-t1	Status 200, t1_label ∈ {INTRO,BACK,METH,RES,DISC,LIM,CONC}
Contribuciones SciBETO	POST /api/contributions con scibeto-es-t2	Status 200, is_contribution booleano

15.2. Resultado esperado

```
[OK] GET /health
[OK] GET /docs
[OK] GET /openapi.json
[OK] GET /api/models
[OK] GET /api/models/scibeto-es-t2/info
[OK] GET /api/lexicon
[OK] GET /api/few_shots
[OK] POST /api/segment (heuristic)
[OK] POST /api/segment (scibeto-es-t1)
[OK] POST /api/contributions (scibeto-es-t2)
```

```
=====
10/10 tests passed · 0 fails
=====
```

Si algún test falla, el script imprime el `curl` exacto que se puede repetir manualmente para diagnosticar.

16. Solución de problemas

Problema	Causa probable	Solución
<code>ModuleNotFoundError: bitsandbytes</code>	Wheel incompatible con la plataforma	<code>pip install bitsandbytes --upgrade</code> o desactivar Llama con <code>LLAMA_ENABLED=false</code>
<code>PermissionError: /app/...</code>	<code>MODELS_DIR</code> apunta a /app/ y no existe	Comentar <code>MODELS_DIR</code> en <code>.env</code> (default infiere desde <code>\$(pwd)/models</code>)
<code>Timeout</code> durante descarga	Red lenta o cuota Drive	Aumentar <code>DOWNLOAD_TIMEOUT_SECONDS=1200</code> y reintentar
<code>OSError: model.safetensors not found</code>	Descarga incompleta	Borrar <code>models/<id>/</code> y volver a correr <code>scripts/download_models.sh</code>
<code>401 Unauthorized</code> al cargar Llama	<code>HF_TOKEN</code> inválido o licencia no aceptada	Aceptar licencia en HuggingFace y regenerar token
<code>429 Too Many Requests</code>	Rate limit (1.000 req/h por IP)	Esperar 1 hora o aumentar <code>RATE_LIMIT_PER_HOUR</code> en <code>.env</code>
<code>502 Bad Gateway</code> desde nginx	El backend uvicorn no responde	<code>sudo systemctl restart maia</code> y revisar <code>journalctl -u maia -n 200</code>

Problema	Causa probable	Solución
OOM al cargar Llama	VRAM insuficiente	Reducir <code>T1_BATCH_SIZE</code> y <code>T2_BATCH_SIZE</code> a 8, o desactivar Llama

16.1. Caso crítico — disco lleno → cascada de fallos

Síntoma: el frontend muestra error genérico de carga y el health check retorna 5xx o no responde.

Causa raíz: el volumen EBS donde corre la app (`/data` o `/`) se llenó, típicamente por:

- Logs no rotados (`journalctl` sin retención, predicciones logueadas con `T1_LOG_PREDICTIONS=true`)
- Cache de Hugging Face creciendo sin límite (`HF_HOME`)
- Descargas de pesos parciales que no se limpiaron
- Tracebacks de Python persistidos por `systemd`

Efecto en cadena:

1. El backend FastAPI no puede escribir logs ni crear archivos temporales → empieza a tirar excepciones I/O
2. El proceso uvicorn entra en estado degradado o crashea
3. `systemd` intenta reiniciar (según política `Restart=on-failure`), pero al no haber espacio el reinicio también falla
4. `nginx` recibe `Bad Gateway` del backend caído → propaga 502 al cliente
5. El frontend (que es estático y se sirve desde `nginx`) sigue cargando, pero todas las llamadas a `/api/*` fallan

Diagnóstico rápido:

```
df -h                                # ¿qué volumen está al 100%?
du -sh /home/ec2-user/app/* | sort -h # ¿qué directorios pesan más?
sudo journalctl --disk-usage          # ¿cuánto consume journalctl?
ls -lh ~/.cache/huggingface          # ¿se desbordó el cache HF?
```

Mitigación:

```
# 1. Liberar logs antiguos de systemd
sudo journalctl --vacuum-time=7d      # mantener solo última semana
sudo journalctl --vacuum-size=500M    # o limitar a 500 MB

# 2. Limpiar cache HF si creció sin control
rm -rf ~/.cache/huggingface/hub/.locks
rm -rf ~/.cache/huggingface/hub/models--*/blobs/* # ¡cuidado, re-descarga!

# 3. Limpiar ZIPs descargados que ya se extrajeron
rm -f /home/ec2-user/app/models/*.zip

# 4. Reiniciar el servicio
sudo systemctl restart maia
df -h                                # validar que haya >5 GB libres
```

Prevención (recomendado):

- Limitar `journalctl` en `/etc/systemd/journald.conf`:

```
SystemMaxUse=500M
SystemMaxFileSize=50M
```

- Definir `HF_HOME=/data/app/models/.hf_cache` y monitorear su tamaño
- Mantener `T1_LOG_PREDICTIONS=false` y `T2_LOG_PREDICTIONS=false` en producción
- Provisionar EBS con al menos **70 GiB** (no 60 GiB), dejando margen operativo

16.2. Logs y diagnóstico

Comando	Para qué
<code>sudo systemctl status maia</code>	Estado actual del servicio
<code>sudo journalctl -u maia -f</code>	Logs en vivo
<code>sudo journalctl -u maia -n 200 --no-pager</code>	Últimas 200 líneas
<code>sudo journalctl -u maia --since "1 hour ago"</code>	Logs de la última hora
<code>curl -s http://localhost:8000/health</code>	Health check local (sin pasar por nginx)
<code>sudo nginx -t</code>	Validar config de nginx
<code>sudo tail -f /var/log/nginx/error.log</code>	Logs de nginx

17. Créditos y agradecimientos

Equipo de desarrollo

Grupo 2 — MAIA 2026:

Integrante	Correo institucional
Santiago Ballesteros Pastrán	s.ballesteros@uniandes.edu.co
Edwin Alexander Cifuentes Bastidas	ea.cifuentes@uniandes.edu.co
Natalia Moncada Suárez	n.moncadas@uniandes.edu.co
Cristian Alexis Murillo Martínez	ca.murillom12@uniandes.edu.co
Juan Pablo Pérez Atehortua	jp.pereza12@uniandes.edu.co

Asesoría académica

- **Rubén Manrique, Ph.D.** — Asesor de proyecto, Universidad de los Andes
- Grupo de Investigación **FLAG — TICsW**, Departamento de Ingeniería de Sistemas y Computación

Recursos externos utilizados

- `Flaglab/SciBETO-large` — modelo base de los encoders fine-tuned
- `meta-llama/Llama-3.1-8B-Instruct` — LLM open-weight de Meta AI
- `gpt-4o-mini` — LLM API de OpenAI
- Corpus **CORE** — proveedor del corpus crudo de ~1.8M documentos en español

Documento académico relacionado

El artículo científico que documenta este trabajo está en [docs/Articulo_PLN_Grupo2_2026.pdf](#) y debe citarse como:

Ballesteros, S., Cifuentes, E. A., Moncada, N., Murillo, C. A., & Pérez, J. P. (2026). *Segmentación Retórica y Detección de Contribuciones en Artículos Científicos en Español: Un Enfoque Comparativo entre Encoders Especializados y Grandes Modelos de Lenguaje*. Proyecto de Grado, Maestría en Inteligencia Artificial, Universidad de los Andes.

18. Licencia

Código

El código fuente de la aplicación ([app/backend/](#), [app/frontend/](#), [app/scripts/](#)) está disponible bajo **licencia MIT**. Ver archivo [LICENSE](#) en la raíz.

Datos

Los Gold Test Sets liberados en [Datos/](#) están bajo **Creative Commons BY 4.0** ([texto completo](#)):

-  Uso libre, incluso comercial
-  Modificación y redistribución permitidas
-  **Atribución obligatoria** al Grupo 2 MAIA 2026 y al artículo asociado

Modelos

Los pesos SciBETO fine-tuned ([scibeto-es-t1](#), [scibeto-es-t2](#), [scibeto-es-t2-ctx](#)) heredan la licencia del modelo base [Flaglab/SciBETO-large](#). Llama 3.1 8B está sujeto a la [Llama 3.1 Community License](#) de Meta.

Artículo científico

El artículo en [docs/Articulo_PLN_Grupo2_2026.pdf](#) es propiedad académica del Grupo 2 y la Universidad de los Andes. Citarse según referencia en §17.

Última actualización del README: 20 de mayo de 2026

Grupo 2 · MAIA 2026 · Universidad de los Andes